



İŞLETİM SİSTEMLERİ

Dönem Projesi

İçerik

İŞLETİM SİSTEMLERİ	1
Dönem Projesi	1
ProcX - Gelişmiş Süreç Yönetim Sistemi	3
1.  PROJE TANIMI	3
2. Proje Hedefleri	3
3.  ÖĞRENİM ÇIKTILARI	3
4.  PROJE YAPISI	3
5.  SİSTEM MİMARİSİ	4
5.1. Veri Yapıları	4
5.2. IPC Mekanizmaları	4
5.3. Thread Yapısı	4
6.  FONKSİYONEL GEREKSİNİMLER	4
6.1. Process Başlatma (fork + exec)	4
6.2. Process Listeleme	5
6.3. Process Sonlandırma	5
6.4. Monitor Thread	6
6.5. IPC Listener Thread	6
6.6. Çıkış ve Temizlik	6
7. Test Senaryoları	7
7.1. Test 1: Tek Instance - Process Başlatma ve Listeleme	7
7.2. Test 2: Çoklu Instance - IPC Testi	7
7.3. Test 3: Attached vs Detached Modu	8
7.4. Test 4: Process Sonlandırma	9
7.5. Test 5: Monitor Thread Testi	9
8.  DEĞERLENDİRME KRİTERLERİ	10
8.1. Detaylı Değerlendirme	10
9.  PROJE TESLİMİ	11
9.1. Genel Kurallar	11
9.2. Teslim Edilecek Dosyalar	12
9.3. Dosya İsimlendirme	12
9.4. Teslim Yöntemi	12
9.5. Son Teslim Tarihi	12

ProcX - Gelişmiş Süreç Yönetim Sistemi

1. PROJE TANIMI

ProcX, Linux ortamında çalışan, kullanıcıların programları arka planda yönetmesine olanak sağlayan ****nohup benzeri**** bir süreç yönetim sistemidir. Birden fazla terminal oturumundan erişilebilir ve tüm instance'lar birbirleriyle senkronize çalışır.

2. Proje Hedefleri

- Process oluşturma ve yönetimi (fork, exec)
- Thread programlama
- Süreçler arası iletişim (IPC)
- Senkronizasyon mekanizmaları
- Sinyal yönetimi

3. ÖĞRENİM ÇIKTILARI

Bu projeyi tamamladığinizda şunları öğrenmiş olacaksınız:

1.  ****fork()**** ile yeni process oluşturma
2.  ****exec()**** ailesi ile program çalıştırma
3.  ****pthread**** ile thread yönetimi
4.  ****Shared Memory**** ile veri paylaşımı
5.  ****Semaphore**** ile senkronizasyon
6.  ****Message Queue**** ile mesajlaşma
7.  ****Signal**** yönetimi (SIGCHLD, SIGINT, SIGTERM)
8.  Daemon process oluşturma

4. PROJE YAPISI

```
procx/
├── procx.c # Ana kaynak kod dosyası (TEK DOSYA)
├── Makefile # Derleme dosyası
└── README.md # Proje açıklaması
```

5. 🗝️ SİSTEM MİMARİSİ

5.1. Veri Yapıları

```
// Process bilgisi
typedef struct {
    pid_t pid; // Process ID
    pid_t owner_pid; // Başlatan instance'ın PID'si
    char command[256]; // Çalıştırılan komut
    ProcessMode mode; // Attached (0) veya Detached (1)
    ProcessStatus status; // Running (0) veya Terminated (1)
    time_t start_time; // Başlangıç zamanı
    int is_active; // Aktif mi? (1: Evet, 0: Hayır)
} ProcessInfo;

// Paylaşılan bellek yapısı
typedef struct {
    ProcessInfo processes[50]; // Maksimum 50 process
    int process_count; // Aktif process sayısı
} SharedData;

// Mesaj yapısı
typedef struct {
    long msg_type; // Mesaj tipi
    int command; // Komut (START/TERMINATE)
    pid_t sender_pid; // Gönderen PID
    pid_t target_pid; // Hedef process PID
} Message;
```

5.2. IPC Mekanizmaları

Mekanizma	İsim	Kullanım Amacı
Shared Memory	/procx_shm	Process listesini tüm instance'lar arasında paylaşma
Semaphore	/procx_sem	Shared memory erişimini senkronize etme
Message Queue	/procx_mq	Instance'lar arası bildirim gönderme

5.3. Thread Yapısı

Her procx instance'ı 3 thread çalıştırır:

1. **Main Thread:** Kullanıcı arayüzü ve menü yönetimi
2. **Monitor Thread:** Process'lerin durumunu izler (her 2 saniyede)
3. **IPC Listener Thread:** Diğer instance'lardan gelen mesajları dinler

6. 🚀 FONKSİYONEL GEREKSİNİMLER

6.1. Process Başlatma (fork + exec)

Gereksinimler:

- Kullanıcıdan komut alınmalı (örn: sleep 100, firefox)

- İki mod desteklenmeli:
- Attached (0): Terminal kapanınca process sonlanır
- Detached (1): Terminal kapansa bile çalışmaya devam eder
- fork() ile yeni process oluşturulmalı
- execvp() ile komut çalıştırılmalı
- Detached modda setsid() ile yeni session oluşturulmalı
- Process bilgisi shared memory'ye eklenmelidir
- Diğer instance'lara message queue ile bildirim gönderilmelidir

Örnek Kullanım:

ProcX v1.0
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiminiz: 1
 Çalıştırılacak komutu girin: sleep 100
 Mod seçin (0: Attached, 1: Detached): 1
 [SUCCESS] Process başlatıldı: PID 12345

6.2. Process Listeleme

Gereksinimler:

- Shared memory'den tüm aktif process'ler okunmalı
- Tablo formatında gösterilmeli:
- PID
- Komut
- Mod (Attached/Detached)
- Owner PID (Hangi instance başlattı)
- Çalışma süresi (saniye)
- Semaphore ile senkronize erişim sağlanmalı

Örnek Çıktı:

ÇALIŞAN PROGRAMLAR				
PID	Command	Mode	Owner	Süre
12345	sleep 100	Detached	10000	15s
12346	firefox	Attached	10001	30s

Toplam: 2 process

6.3. Process Sonlandırma

Gereksinimler:

- Kullanıcıdan PID alınmalı
- kill(pid, SIGTERM) ile process sonlandırılmalı

- Shared memory'den process bilgisi silinmeli
- Diğer instance'lara bildirim gönderilmeli

Örnek Kullanım:

ProcX v1.0
1. Yeni Program Çalıştır 2. Çalışan Programları Listele 3. Program Sonlandır 0. Çıkış

Seçiminiz: 3

Sonlandırılacak process PID: 12345

[INFO] Process 12345'e SIGTERM sinyali gönderildi

6.4. Monitor Thread

Gereksinimler:

- Her 2 saniyede bir çalışmalı
- waitpid(pid, &status, WNOHANG) ile process durumunu kontrol etmeli
- Sonlanan process'leri shared memory'den temizlemeli
- Diğer instance'lara bildirim göndermeli

Örnek Çıktı:

[MONITOR] Process 12345 sonlandı

6.5. IPC Listener Thread

Gereksinimler:

- Message queue'yu sürekli dinlemeli
- Diğer instance'lardan gelen mesajları işlemeli
- Kendi gönderdiği mesajları atlamalı (sender_pid == getpid())
- Bildirimleri ekrana yazdırmalı

Örnek Çıktı:

[IPC] Yeni process başlatıldı: PID 12345

[IPC] Process sonlandırıldı: PID 12345

6.6. Çıkış ve Temizlik

Gereksinimler:

- Kullanıcı 0 seçtiğinde program sonlanmalı
- Tüm thread'ler düzgün şekilde kapatılmalı (pthread_join)
- Attached modda çalışan ve bu instance tarafından başlatılan tüm process'ler sonlandırılmalı
- Detached modda çalışan process'ler çalışmaya devam etmeli
- IPC kaynakları temizlenmeli (close, unmap)

7. Test Senaryoları

7.1. Test 1: Tek Instance - Process Başlatma ve Listeleme

Terminal 1:

```
$ ./procx
```

ProcX v1.0
1. Yeni Program Çalıştır 2. Çalışan Programları Listele 3. Program Sonlandır 0. Çıkış

Seçiminiz: 1

Çalıştırılacak komutu girin: sleep 100

Mod seçin (0: Attached, 1: Detached): 1

[SUCCESS] Process başlatıldı: PID 12345

ProcX v1.0
1. Yeni Program Çalıştır 2. Çalışan Programları Listele 3. Program Sonlandır 0. Çıkış

Seçiminiz: 2

Process listesini görmelisiniz

ProcX v1.0
1. Yeni Program Çalıştır 2. Çalışan Programları Listele 3. Program Sonlandır 0. Çıkış

Seçiminiz: 0

Program sonlanmalı, detached process çalışmaya devam etmeli

7.2. Test 2: Çoklu Instance - IPC Testi

Terminal 1:

```
$ ./procx
```

Terminal 2:

```
$ ./procx
```

Terminal 1'de:

ProcX v1.0
1. Yeni Program Çalıştır 2. Çalışan Programları Listele 3. Program Sonlandır 0. Çıkış

```
Seçiminiz: 1
Çalıştırılacak komutu girin: sleep 200
Mod seçin (0: Attached, 1: Detached): 1
[SUCCESS] Process başlatıldı: PID 12346

# Terminal 2'de otomatik olarak görmelisiniz:
[IPC] Yeni process başlatıldı: PID 12346
```

Terminal 2'de:

ProcX v1.0
1. Yeni Program Çalıştır 2. Çalışan Programları Listele 3. Program Sonlandır 0. Çıkış

```
Seçiminiz: 2
# Her iki terminal'den başlatılan process'leri görmelisiniz
```

7.3. Test 3: Attached vs Detached Modu

Terminal 1:

```
$ ./procx
```

ProcX v1.0
1. Yeni Program Çalıştır 2. Çalışan Programları Listele 3. Program Sonlandır 0. Çıkış

```
Seçiminiz: 1
Çalıştırılacak komutu girin: sleep 300
Mod seçin (0: Attached, 1: Detached): 0
[SUCCESS] Process başlatıldı: PID 12347
```

ProcX v1.0
1. Yeni Program Çalıştır 2. Çalışan Programları Listele 3. Program Sonlandır 0. Çıkış

```
Seçiminiz: 1
Çalıştırılacak komutu girin: sleep 400
Mod seçin (0: Attached, 1: Detached): 1
[SUCCESS] Process başlatıldı: PID 12348
```

ProcX v1.0
1. Yeni Program Çalıştır 2. Çalışan Programları Listele 3. Program Sonlandır 0. Çıkış

```
Seçiminiz: 0
# Çıkış yap
```

Başka bir terminal'de kontrol et:

```
$ ps aux | grep sleep
# PID 12347 (attached) sonlanmış olmalı
# PID 12348 (detached) çalışmaya devam etmeli
```

7.4. Test 4: Process Sonlandırma

Terminal 1:

```
$ ./procx
```

ProcX v1.0
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

```
Seçiminiz: 1
Çalıştırılacak komutu girin: sleep 500
Mod seçin (0: Attached, 1: Detached): 1
[SUCCESS] Process başlatıldı: PID 12349
```

ProcX v1.0
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

```
Seçiminiz: 3
Sonlandırılacak process PID: 12349
[INFO] Process 12349'e SIGTERM sinyali gönderildi
```

```
# Birkaç saniye sonra:
[MONITOR] Process 12349 sonlandı
```

7.5. Test 5: Monitor Thread Testi

Terminal 1:

```
$ ./procx
```

ProcX v1.0
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

```
Seçiminiz: 1
Çalıştırılacak komutu girin: sleep 5
```

```
Mod seçin (0: Attached, 1: Detached): 1
[SUCCESS] Process başlatıldı: PID 12350

# 5 saniye bekleyin, otomatik olarak görmelisiniz:
[MONITOR] Process 12350 sonlandı
```

8. DEĞERLENDİRME KRİTERLERİ

Kriter	Puan	Açıklama
Fork Kullanımı	20	fork() ile process oluşturma, parent-child ilişkisi
Exec Kullanımı	20	execvp() ile program çalıştırma, komut parse etme
Thread Yönetimi	20	Monitor ve IPC thread'lerinin doğru çalışması
Shared Memory	15	Process listesinin paylaşılması, doğru okuma/yazma
Semaphore	10	Race condition önleme, senkronizasyon
Message Queue	10	Instance'lar arası mesajlaşma
Kod Kalitesi	5	Temiz kod, yorum satırları, hata yönetimi
TOPLAM	100	

8.1. Detaylı Değerlendirme

Fork Kullanımı (20 puan)

- ✓ fork() doğru kullanılmış (5 puan)
- ✓ Parent-child ayrımı yapılmış (5 puan)
- ✓ Hata kontrolü var (5 puan)
- ✓ Zombie process önleme (5 puan)

Exec Kullanımı (20 puan)

- ✓ execvp() doğru kullanılmış (5 puan)
- ✓ Komut parse edilmiş (5 puan)
- ✓ Hata kontrolü var (5 puan)
- ✓ Detached mod için setsid() kullanılmış (5 puan)

Thread Yönetimi (20 puan)

- ✓ Monitor thread çalışıyor (7 puan)
- ✓ IPC listener thread çalışıyor (7 puan)
- ✓ Thread'ler düzgün kapatılıyor (6 puan)

Shared Memory (15 puan)

- ✓ Shared memory oluşturulmuş (5 puan)
- ✓ Doğru okuma/yazma (5 puan)
- ✓ Temizlik yapılıyor (5 puan)

Semaphore (10 puan)

- ✓ Semaphore kullanılmış (5 puan)
- ✓ Race condition önlenmiş (5 puan)

Message Queue (10 puan)

- ✓ Message queue kullanılmış (5 puan)
- ✓ Mesajlar doğru gönderiliyor/alınıyor (5 puan)

Kod Kalitesi (5 puan)

- ✓ Temiz ve okunabilir kod (2 puan)
- ✓ Yorum satırları var (2 puan)
- ✓ Hata yönetimi iyi (1 puan)

9. PROJE TESLİMİ

9.1. Genel Kurallar

- **Programlama Dili:**
 - Proje SADECE C dilinde yazılmalıdır. Başka bir programlama dilinde (C++, Python, Java vb.) yapılan projeler KABUL EDİLMEYECEKTİR.
- **Bilgi Birikimi:**
 - Projenin tamamını çözebilecek bilgi birikimine ve araştırma-öğrenme yetisine sahipsiniz. İşletim sistemi dersinde işlenen konular (fork, exec, thread, IPC, semaphore, shared memory) göz önünde bulundurularak yazılmalıdır.
- **Rapor Formatı:**
 - Proje teslimi için verilen rapor formatını DÜZGÜN ve ÖZENLİ bir şekilde yazınız (puanlama kısmında var).
- **Kod Kalitesi:**
 - Proje kod kısmını temel programlama kurallarına uyarak ve açıklama satırları ile birlikte yazınız (puanlama kısmında var).
- **Makefile:**
 - Projede yer alan program kodlarını (c dosyaları) ile bir adet makefile dosyası oluşturunuz. Makefile dosyası çalıştırıldığında program otomatik derlenebilmelidir. Ödev kontrolcüsü sadece makefile dosyasını çalıştıracak ve sonra terminalden `procx` programını çalıştıracaktır. Makefile dosyası olmayan projeler değerlendirilmez.

9.2. Teslim Edilecek Dosyalar

Gönderilecek proje dosyası içinde yer alacak belgeler:

```
adi_soyadi_no_opsis_proj/
├── makefile # Derleme dosyası (ZORUNLU)
├── procx.c # Ana kaynak kod dosyası (ZORUNLU)
├── rapor.pdf # Proje raporu (ZORUNLU)
└── ekran_ciktilari/ # Ekran görüntüleri klasörü (ZORUNLU)
    ├── test1.png
    ├── test2.png
    ├── test3.png
    └── ...
```

9.3. Dosya İsimlendirme

- Proje klasörünü ****adi_soyadi_no_opsis_proje**** olarak isimlendiriniz
- Aynı isimde ****ZIP**** formatında sıkıştırınız
- Örnek: `ahmet\_yilmaz\_123456\_opsis\_proj.zip`

9.4. Teslim Yöntemi

Projeler lms üzerinden alınacaktır.

9.5. Son Teslim Tarihi

16 Aralık 2024 - 23:00

⚠ ****Projenin bütünlemesi OLMAYACAKTIR.**** Son teslim tarihini kaçırmayınız!